

# 计算机系统结构实验报告Lab03:

## Ctrl, ALU Ctrl, ALU

uu-matter543

2024-2025-2

### 摘要

在 Lab03 中，我成功实现了 MIPS 32 单周期处理器的 3 个部件：Control Unit, ALU Control Unit, ALU，学习并使用了 Verilog 的 case 语句，初步了解了 CPU 的模块化设计以及利用状态机思想设计 CPU 模块，收获颇丰。

### 目录

|          |                                 |           |
|----------|---------------------------------|-----------|
| <b>1</b> | <b>实验目的</b>                     | <b>2</b>  |
| <b>2</b> | <b>原理分析</b>                     | <b>2</b>  |
| 2.1      | Control Unit 原理分析 . . . . .     | 2         |
| 2.2      | ALU Control Unit 功能分析 . . . . . | 3         |
| 2.3      | ALU 功能分析 . . . . .              | 3         |
| <b>3</b> | <b>功能实现</b>                     | <b>4</b>  |
| 3.1      | Control Unit 功能实现 . . . . .     | 4         |
| 3.2      | ALU Control Unit 功能实现 . . . . . | 5         |
| 3.3      | ALU 功能实现 . . . . .              | 5         |
| <b>4</b> | <b>功能仿真</b>                     | <b>6</b>  |
| 4.1      | Control Unit 功能仿真 . . . . .     | 6         |
| 4.2      | ALU Control Unit 功能仿真 . . . . . | 7         |
| 4.3      | ALU 功能仿真 . . . . .              | 9         |
| <b>5</b> | <b>收获与反思</b>                    | <b>10</b> |

## 1 实验目的

1. 理解主控制部件或单元、ALU 控制器单元、ALU 单元的原理；
2. 熟悉所需的 MIPS 32 指令集；
3. 使用 Verilog HDL 设计与实现主控制器部件 (Ctrl)；
4. 使用 Verilog HDL 设计与实现 ALU 控制器部件 (ALUCtrl)；
5. 使用 Verilog HDL 设计与实现 ALU 部件 (ALU)；
6. 使用Vivado进行功能模块的行为仿真。

## 2 原理分析

### 2.1 Control Unit 原理分析

MIPS 32 指令集分为 3 种：R-type, I-type, J-type. Control Unit 的功能是根据 32 位指令的最高 6 位 opcode 进行译码，并输出对应的控制信号。控制信号如表所示：

| 信号名         | 功能                                                               |
|-------------|------------------------------------------------------------------|
| reg_dst     | 选择寄存器写操作的目标寄存器<br>0 为指令的 [20:16] 位，1为指令的 [15:11] 位               |
| alu_src     | 选择 ALU 的第 2 个操作数<br>0 为指令的 [20:16] 位表示的寄存器，1 为指令的 [15:0] 位表示的立即数 |
| mem_to_reg  | 选择寄存器写操作的数据来源<br>0 为 ALU 的运算结果，1 为 Data Memory 的对应地址             |
| reg_write   | 寄存器写操作使能信号                                                       |
| mem_read    | Data Memory 读操作使能信号                                              |
| mem_write   | Data Memory 写操作使能信号                                              |
| branch      | 条件跳转指令使能信号                                                       |
| alu_op[1:0] | 输入到 ALU Control Unit，用于进一步处理                                     |
| jump        | 无条件跳转指令使能信号                                                      |

opcode 与 Control Unit 输出的关系如表所示：

| 指令类型<br>opcode | R-type<br>000000 | J-type<br>000010 | I-type beq<br>000100 | I-type lw<br>100011 | I-type sw<br>101011 |
|----------------|------------------|------------------|----------------------|---------------------|---------------------|
| reg_dst        | 1                | 0                | 0                    | 0                   | 0                   |
| alu_src        | 0                | 0                | 0                    | 1                   | 1                   |
| mem_to_reg     | 0                | 0                | 0                    | 1                   | 0                   |
| reg_write      | 1                | 0                | 0                    | 0                   | 0                   |
| mem_read       | 0                | 0                | 0                    | 1                   | 0                   |
| mem_write      | 0                | 0                | 0                    | 0                   | 1                   |
| branch         | 0                | 0                | 1                    | 0                   | 0                   |
| alu_op         | 10               | 00               | 01                   | 00                  | 00                  |
| jump           | 0                | 1                | 0                    | 0                   | 0                   |

## 2.2 ALU Control Unit 功能分析

ALU Control Unit 的功能是根据 Control Unit 的输出 `alu_op` 和 R-type 指令中的最低 6 位 `funct` 字段，进行相应的译码，输出 `alu_ctr_out`，以控制 ALU 进行各种运算。

指令名称，输入 `alu_op`, `funct`，输出 `alu_ctr_out`，执行运算的关系如表所示：

| 指令    | alu_op | funct   | alu_ctr_out | 执行运算 |
|-------|--------|---------|-------------|------|
| lw/sw | 00     | xxxxxxx | 0010        | ADD  |
| beq   | 01     | xxxxxxx | 0110        | SUB  |
| add   | 10     | 100000  | 0010        | ADD  |
| sub   | 10     | 100010  | 0110        | SUB  |
| and   | 10     | 100100  | 0000        | AND  |
| or    | 10     | 100101  | 0001        | OR   |
| slt   | 10     | 101010  | 0111        | NOR  |

## 2.3 ALU 功能分析

ALU 的功能是根据 ALU Control Unit 的输出 `alu_ctr` 和 2 个 32 位操作数，进行相应运算，并输出结果 `alu_result`；同时 ALU 还有 Zero Flag，其在运算结果为 0 时置 1，运算结果不为 0 时置 0。

输入 `alu_ctr`、输出 `alu_result` 的关系如表所示：

| 指令    | alu_ctr | 执行运算        |
|-------|---------|-------------|
| lw/sw | 0010    | OP1+OP2     |
| beq   | 0110    | OP1-OP2     |
| add   | 0010    | OP1+OP2     |
| sub   | 0110    | OP1-OP2     |
| and   | 0000    | OP1 AND OP2 |
| or    | 0001    | OP1 OR OP2  |
| slt   | 0111    | OP1 NOR OP2 |

### 3 功能实现

#### 3.1 Control Unit 功能实现

按照功能分析中的输入 `op_code` 和输出信号列表定义端口，而从输入到输出的逻辑可用 `case` 语句实现分支功能，类似于 C/C++ 中的 `switch case` 结构，实现如下：

```
module control_unit(  
    input  [5:0] op_code ,  
    output reg reg_dst ,  
    output reg alu_src ,  
    output reg mem_to_reg ,  
    output reg reg_write ,  
    output reg mem_read ,  
    output reg mem_write ,  
    output reg branch ,  
    output reg [1:0] alu_op ,  
    output reg jump  
);  
always @(op_code) begin  
    case (op_code)  
        6'b000000: begin // R ...  
        6'b100011: begin // lw ...  
        6'b101011: begin // sw ...  
        6'b000100: begin // beq ...  
        6'b000010: begin // J ...  
        default: begin // ...  
    endcase  
end
```

```
endmodule
```

6 位 `op_code` 分别对应的指令类型如代码中注释所示。 `default` 表示空指令，此时控制信号输出全为 0，表示不改变机器状态。其余情况各控制信号参照功能表依次赋值即可。

其中输出信号须定义为 `reg` 类型，以使控制信号能够持续输出，不能完全按照实验指导书上的示例代码编写。这里直接将 `reg` 类型声明嵌入端口定义处，如端口定义代码所示。

## 3.2 ALU Control Unit 功能实现

按照输入 `op_code` 和 `funct`，输出 `alu_ctr_out` 定义端口；在 ALU 控制单元的表中，有些位是无关位，即该位值为 0 或 1 都应得到对应结果，用 `x` 表示；实现时，可以用 `casex` 语句代替 `case` 语句，两种语句唯一的差别是 `casex` 语句将 `x` 视为无关位，实现如下：

```
module alu_control_unit(  
    input [5:0] funct,  
    input [1:0] alu_op,  
    output reg [3:0] alu_ctr_out  
);  
always @(alu_op or funct) begin  
    casex ({alu_op, funct})  
        8'b00xxxxxx: alu_ctr_out = 4'b0010;  
        8'b01xxxxxx: alu_ctr_out = 4'b0110;  
        8'b1xxx0000: alu_ctr_out = 4'b0010;  
        8'b1xxx0010: alu_ctr_out = 4'b0110;  
        8'b1xxx0100: alu_ctr_out = 4'b0000;  
        8'b1xxx0101: alu_ctr_out = 4'b0001;  
        8'b1xxx1010: alu_ctr_out = 4'b0111;  
        default: alu_ctr_out = 4'b0;  
    endcase  
end  
endmodule
```

## 3.3 ALU 功能实现

ALU 的实现比较简单，可直接使用 `case` 语句，根据控制信号进行运算即可。其中，`slt` 指令是有符号数的比较，需要使用 `$signed()` 将寄存器值视为有符号数。实现如下：

```
module alu(  
    input [31:0] reg_1,  
    input [31:0] reg_2,
```

```
input [3:0] alu_ctr,
output reg zero,
output reg [31:0] alu_result
);
always @(reg_1 or reg_2 or alu_ctr) begin
    case (alu_ctr)
        4'b0000: alu_result = reg_1 & reg_2;
        4'b0001: alu_result = reg_1 | reg_2;
        4'b0010: alu_result = reg_1 + reg_2;
        4'b0110: alu_result = reg_1 - reg_2;
        4'b0111: alu_result = $signed(reg_1) < $signed(reg_2) ? 1
            : 0;
        4'b1100: alu_result = ~(reg_1 | reg_2);
    endcase
    if (alu_result == 0) zero = 1;
    else zero = 0;
end
endmodule
```

## 4 功能仿真

注意在进行功能仿真时，要根据需要进行仿真的模块，将对应的仿真激励 Set as Top，具体操作在实验指导书中有详细说明，这里不再赘述。

### 4.1 Control Unit 功能仿真

编写激励文件 control\_unit\_tb.v，连接仿真激励输入与模块输入，在每个要仿真的功能之间添加时延，以便在波形图上观察到输出信号的变化。主要的连接和仿真激励如下：

```
control_unit example_ctrlu (
    .op_code(OpCode),
    .reg_dst(RegDst),
    .alu_src(ALUSrc),
    .mem_to_reg(MemToReg),
    .reg_write(RegWrite),
    .mem_read(MemRead),
    .mem_write(MemWrite),
    .branch(Branch),
```

```

    .alu_op(ALUOp),
    .jump(Jump)
);

initial begin
    OpCode = 0;
    #100;
    #100 OpCode = 6'b000000; // R type
    #100 OpCode = 6'b100011; // lw
    #100 OpCode = 6'b101011; // sw
    #100 OpCode = 6'b000100; // beq
    #100 OpCode = 6'b000010; // J type
    #100 OpCode = 6'b111111; // default
end

```

仿真激励依次输入了 R-type, lw, sw, beq, J-type的op\_code 控制字, 输出波形应与 opcode 与 Control Unit 输出的关系表相对应, 波形如下:

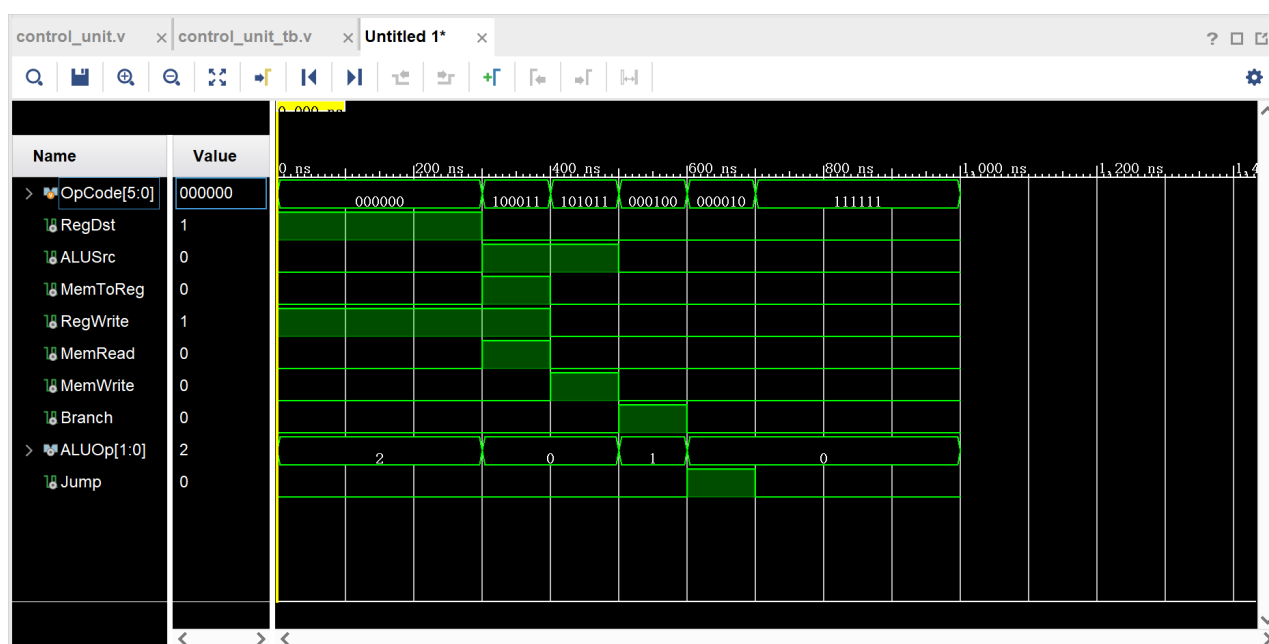


图 1: Control Unit 功能仿真

核对无误, 说明 Control Unit 功能基本实现。

## 4.2 ALU Control Unit 功能仿真

编写激励文件 alu\_control\_unit\_tb.v, 连接仿真激励输入与模块输入, 在每个要仿真的功能之间添加时延, 以便在波形图上观察到输出信号的变化, 主要的仿真激励如下:

```

alu_control_unit example_aluctrlu (
    .funct(Funct),
    .alu_op(ALUOp),
    .alu_ctr_out(ALUCtrOut)
);

initial begin
    Funct = 0;
    ALUOp = 0;
    #100;
    #100 begin ALUOp = 2'b00; Funct = 8'b000000; end
    #100 begin ALUOp = 2'b01; Funct = 8'b000000; end
    #100 begin ALUOp = 2'b10; Funct = 8'b000000; end
    #100 begin ALUOp = 2'b10; Funct = 8'b000010; end
    #100 begin ALUOp = 2'b10; Funct = 8'b000100; end
    #100 begin ALUOp = 2'b10; Funct = 8'b000101; end
    #100 begin ALUOp = 2'b10; Funct = 8'b001010; end
end

```

仿真激励依次输入了 lw/sw, beq, add, sub, and, or, slt 指令的 alu\_op 和 funct 控制字。输出波形应与输入 alu\_op, funct, 输出 alu\_ctr\_out 的关系表对应, 波形如下:

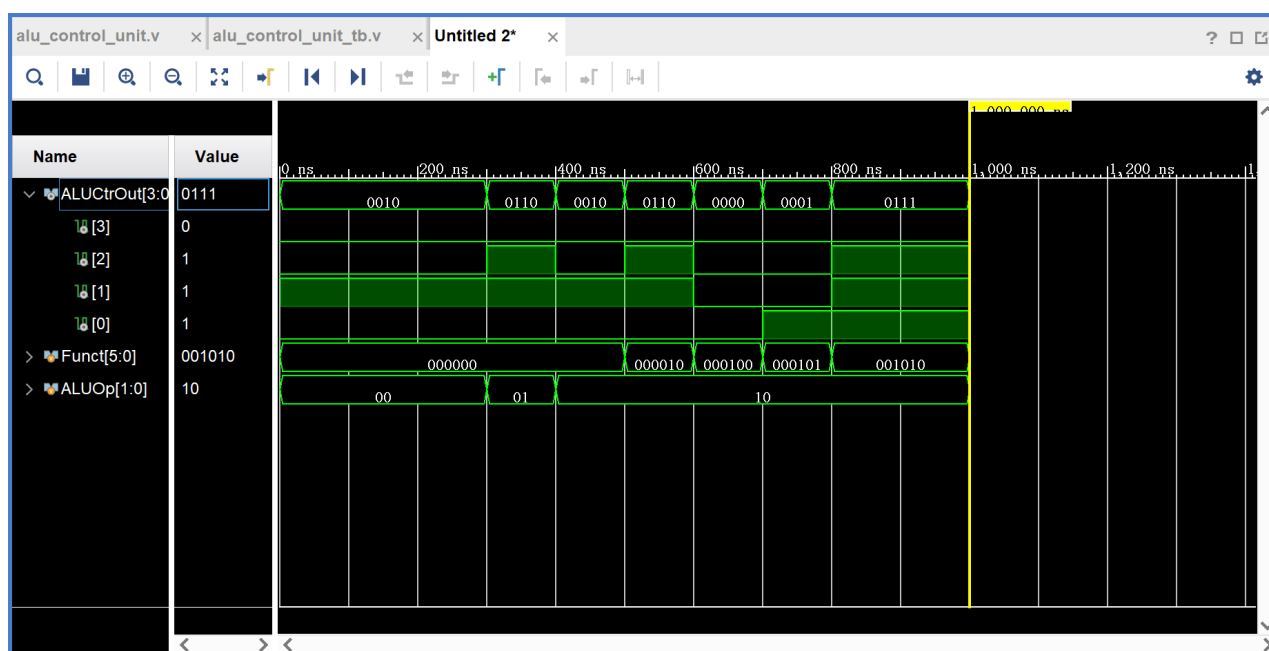


图 2: ALU Control Unit 功能仿真

实验指导书中给出了 2 种波形, 其中图 A 的波形出现了红色部分, 是因为激励文件代码在赋值时写入了 x (无关位)。方便起见, 我在激励文件代码中将无关位均赋值为 0, 因此



得到了如图 B 所示的波形图。核对无误，说明 ALU Control Unit 功能基本实现。

### 4.3 ALU 功能仿真

编写激励文件 alu\_tb.v，连接仿真激励输入与模块输入，在每个要仿真的功能之间添加时延，以便在波形图上观察到输出信号的变化，主要的仿真激励如下：

```
alu example_alu(  
    .reg_1(Operand1),  
    .reg_2(Operand2),  
    .alu_ctr(AluCtr),  
    .zero(Zero),  
    .alu_result(AluResult)  
);  
initial begin  
    Operand1 = 0;  
    Operand2 = 0;  
    AluCtr = 0;  
    #100 begin Operand1 = 15; Operand2 = 10; AluCtr = 4'b0000; end  
    #100 AluCtr = 4'b0001;  
    #100 AluCtr = 4'b0010;  
    #100 AluCtr = 4'b0110;  
    #100 begin Operand1 = 10; Operand2 = 15; end  
    #100 begin AluCtr = 4'b0111; Operand1 = 15; Operand2 = 10; end  
    #100 begin Operand1 = 10; Operand2 = 15; end  
    #100 begin AluCtr = 4'b1100; Operand1 = 1; Operand2 = 1; end  
    #100 Operand1 = 16;  
end
```

仿真激励依次为：

- 0 时刻初始化操作数 1，操作数 2，alu\_ctr 均为 0
- 100 时刻，只看运算有关的低 4 位，计算  $1111 \text{ AND } 1010$ ，结果应为  $(1010)_2 = 10$
- 200 时刻，只看运算有关的低 4 位，计算  $1111b \text{ OR } 1010b$ ，结果应为  $(1111)_2 = 15$
- 300 时刻，计算  $15 + 10$ ，结果应为 25
- 400 时刻，计算  $15 - 10$ ，结果应为 5
- 500 时刻，计算  $10 - 15$ ，结果应为 -5
- 600 时刻，计算  $15 < 10$ ，结果应为 0
- 700 时刻，计算  $10 < 15$ ，结果应为 1
- 800 时刻和 900 时刻，做 NOR 运算，具体可参考 NOR 运算波形。



- wire 类型的输出信号不能持续输出，需声明为 reg 以解决问题；
- 仿真时直接令激励为 x 会让波形图中出现红色部分；
- slt 指令实现时需要注意有符号比较和无符号比较。

在查阅 Verilog 相关资料、询问他人后，我成功解决了以上问题，增进了对 Verilog 的了解，提高了使用 Vivado 进行硬件开发的熟练度。